

Submission Worksheet

Submission Data

Course: IT202-008-S2026

Assignment: IT202 Milestone 1 - 2026

Student: Deep S. (dns33)

Status: Submitted | **Worksheet Progress:** 100%

Potential Grade: 10.00/10.00 (100.00%)

Received Grade: 0.00/10.00 (0.00%)

Started: 4/9/2026 6:51:52 PM

Updated: 4/11/2026 2:05:15 PM

Grading Link: <https://learn.ethereallab.app/assignment/v3/IT202-008-S2026/it202-milestone1-2026/dns33>

View Link: <https://learn.ethereallab.app/assignment/v3/IT202-008-S2026/it202-milestone-1-2026/view/dns33>

Instructions

- Overview Link: <https://youtu.be/IDtqdaLwcVg>

1. Refer to Milestone1 of this doc:

<https://docs.google.com/document/d/1XE96a8DQ52Vp49XACBDTNCq0xYDt3kF29cO88EWVwfo/view>

2. Ensure you read all instructions and objectives before starting.

3. Ensure you've gone through each lesson related to this Milestone

4. Switch to the Milestone1 branch

1. `git checkout Milestone1` (ensure proper starting branch)

2. `git pull origin Milestone1` (ensure history is up to date)

5. Fill out the below worksheet

- Ensure there's a comment with your UCID, date, and brief summary of the snippet in each screenshot
- Ensure proper styling is applied to each page
- Ensure there are no visible technical errors; only user-friendly messages are allowed

6. Once finished, click "Submit and Export"

7. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github

1. `git add .`

2. `git commit -m "adding PDF"`

3. `git push origin Milestone1`

4. On Github merge the pull request from Milestone1 to qa

5. On Github create a pull request from qa to prod and immediately merge. (This will trigger the prod deploy to make the heroku prod links work)

8. Upload the same PDF to Canvas

9. Sync Local

10. `git checkout qa`

11. `git pull origin qa`

Section #1: (2 pts.) Feature: User Will Be Able To Register A New Account

Progress: 100%

≡ Task #1 (0.67 pts.) - Validation

Progress: 100%

Details:

- Show the relevant code snippets for each validation layer
- Show the relevant demo of each validation layer
- Briefly explain how the validation steps work at each layer

≡ Task #1 (0.33 pts.) - HTML Form/Validation

Progress: 100%

Details:

- System should allow the user to correct the error without wiping/clearing the form (for the PHP side this includes sticky-form logic to keep the username/email address entries)

Part 1:

Progress: 100%

Details:

- Show the code related to the register page's form (HTML validation)
- Show examples of each validation message (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)

```
1 public function __construct() {  
2     $this->load->add_js_file('validation.js');  
3 }  
4  
5  
6  
7  
8  
9  
10  
11  
12  
13  
14  
15  
16  
17  
18  
19  
20  
21  
22  
23  
24  
25  
26  
27  
28  
29  
30  
31  
32  
33  
34  
35  
36  
37  
38  
39  
40  
41  
42  
43  
44  
45  
46  
47  
48  
49  
50  
51  
52  
53  
54  
55  
56  
57  
58  
59  
60  
61  
62  
63  
64  
65  
66  
67  
68  
69  
70  
71  
72  
73  
74  
75  
76  
77  
78  
79  
80  
81  
82  
83  
84  
85  
86  
87  
88  
89  
90  
91  
92  
93  
94  
95  
96  
97  
98  
99  
100  
101  
102  
103  
104  
105  
106  
107  
108  
109  
110  
111  
112  
113  
114  
115  
116  
117  
118  
119  
120  
121  
122  
123  
124  
125  
126  
127  
128  
129  
130  
131  
132  
133  
134  
135  
136  
137  
138  
139  
140  
141  
142  
143  
144  
145  
146  
147  
148  
149  
150  
151  
152  
153  
154  
155  
156  
157  
158  
159  
160  
161  
162  
163  
164  
165  
166  
167  
168  
169  
170  
171  
172  
173  
174  
175  
176  
177  
178  
179  
180  
181  
182  
183  
184  
185  
186  
187  
188  
189  
190  
191  
192  
193  
194  
195  
196  
197  
198  
199  
200  
201  
202  
203  
204  
205  
206  
207  
208  
209  
210  
211  
212  
213  
214  
215  
216  
217  
218  
219  
220  
221  
222  
223  
224  
225  
226  
227  
228  
229  
230  
231  
232  
233  
234  
235  
236  
237  
238  
239  
240  
241  
242  
243  
244  
245  
246  
247  
248  
249  
250  
251  
252  
253  
254  
255  
256  
257  
258  
259  
260  
261  
262  
263  
264  
265  
266  
267  
268  
269  
270  
271  
272  
273  
274  
275  
276  
277  
278  
279  
280  
281  
282  
283  
284  
285  
286  
287  
288  
289  
290  
291  
292  
293  
294  
295  
296  
297  
298  
299  
300  
301  
302  
303  
304  
305  
306  
307  
308  
309  
310  
311  
312  
313  
314  
315  
316  
317  
318  
319  
320  
321  
322  
323  
324  
325  
326  
327  
328  
329  
330  
331  
332  
333  
334  
335  
336  
337  
338  
339  
340  
341  
342  
343  
344  
345  
346  
347  
348  
349  
350  
351  
352  
353  
354  
355  
356  
357  
358  
359  
360  
361  
362  
363  
364  
365  
366  
367  
368  
369  
370  
371  
372  
373  
374  
375  
376  
377  
378  
379  
380  
381  
382  
383  
384  
385  
386  
387  
388  
389  
390  
391  
392  
393  
394  
395  
396  
397  
398  
399  
400  
401  
402  
403  
404  
405  
406  
407  
408  
409  
410  
411  
412  
413  
414  
415  
416  
417  
418  
419  
420  
421  
422  
423  
424  
425  
426  
427  
428  
429  
430  
431  
432  
433  
434  
435  
436  
437  
438  
439  
440  
441  
442  
443  
444  
445  
446  
447  
448  
449  
450  
451  
452  
453  
454  
455  
456  
457  
458  
459  
460  
461  
462  
463  
464  
465  
466  
467  
468  
469  
470  
471  
472  
473  
474  
475  
476  
477  
478  
479  
480  
481  
482  
483  
484  
485  
486  
487  
488  
489  
490  
491  
492  
493  
494  
495  
496  
497  
498  
499  
500  
501  
502  
503  
504  
505  
506  
507  
508  
509  
510  
511  
512  
513  
514  
515  
516  
517  
518  
519  
520  
521  
522  
523  
524  
525  
526  
527  
528  
529  
530  
531  
532  
533  
534  
535  
536  
537  
538  
539  
540  
541  
542  
543  
544  
545  
546  
547  
548  
549  
550  
551  
552  
553  
554  
555  
556  
557  
558  
559  
560  
561  
562  
563  
564  
565  
566  
567  
568  
569  
570  
571  
572  
573  
574  
575  
576  
577  
578  
579  
580  
581  
582  
583  
584  
585  
586  
587  
588  
589  
590  
591  
592  
593  
594  
595  
596  
597  
598  
599  
600  
601  
602  
603  
604  
605  
606  
607  
608  
609  
610  
611  
612  
613  
614  
615  
616  
617  
618  
619  
620  
621  
622  
623  
624  
625  
626  
627  
628  
629  
630  
631  
632  
633  
634  
635  
636  
637  
638  
639  
640  
641  
642  
643  
644  
645  
646  
647  
648  
649  
650  
651  
652  
653  
654  
655  
656  
657  
658  
659  
660  
661  
662  
663  
664  
665  
666  
667  
668  
669  
670  
671  
672  
673  
674  
675  
676  
677  
678  
679  
680  
681  
682  
683  
684  
685  
686  
687  
688  
689  
690  
691  
692  
693  
694  
695  
696  
697  
698  
699  
700  
701  
702  
703  
704  
705  
706  
707  
708  
709  
710  
711  
712  
713  
714  
715  
716  
717  
718  
719  
720  
721  
722  
723  
724  
725  
726  
727  
728  
729  
730  
731  
732  
733  
734  
735  
736  
737  
738  
739  
740  
741  
742  
743  
744  
745  
746  
747  
748  
749  
750  
751  
752  
753  
754  
755  
756  
757  
758  
759  
760  
761  
762  
763  
764  
765  
766  
767  
768  
769  
770  
771  
772  
773  
774  
775  
776  
777  
778  
779  
780  
781  
782  
783  
784  
785  
786  
787  
788  
789  
790  
791  
792  
793  
794  
795  
796  
797  
798  
799  
800  
801  
802  
803  
804  
805  
806  
807  
808  
809  
810  
811  
812  
813  
814  
815  
816  
817  
818  
819  
820  
821  
822  
823  
824  
825  
826  
827  
828  
829  
830  
831  
832  
833  
834  
835  
836  
837  
838  
839  
840  
841  
842  
843  
844  
845  
846  
847  
848  
849  
850  
851  
852  
853  
854  
855  
856  
857  
858  
859  
860  
861  
862  
863  
864  
865  
866  
867  
868  
869  
870  
871  
872  
873  
874  
875  
876  
877  
878  
879  
880  
881  
882  
883  
884  
885  
886  
887  
888  
889  
890  
891  
892  
893  
894  
895  
896  
897  
898  
899  
900  
901  
902  
903  
904  
905  
906  
907  
908  
909  
910  
911  
912  
913  
914  
915  
916  
917  
918  
919  
920  
921  
922  
923  
924  
925  
926  
927  
928  
929  
930  
931  
932  
933  
934  
935  
936  
937  
938  
939  
940  
941  
942  
943  
944  
945  
946  
947  
948  
949  
950  
951  
952  
953  
954  
955  
956  
957  
958  
959  
960  
961  
962  
963  
964  
965  
966  
967  
968  
969  
970  
971  
972  
973  
974  
975  
976  
977  
978  
979  
980  
981  
982  
983  
984  
985  
986  
987  
988  
989  
990  
991  
992  
993  
994  
995  
996  
997  
998  
999  
1000
```

code snippet



empty form

A screenshot of a web browser displaying a registration form. The form has four input fields: 'username', 'email', 'password', and 'confirm'. All fields are empty. Below the fields is a blue 'register' button. The browser's address bar shows a URL starting with 'http://localhost:3000/'.

invalid email

A screenshot of a web browser displaying a registration form. The 'email' field contains the text 'invalid@email.com'. A red error message 'Invalid email format' is visible below the field. The 'password' and 'confirm' fields are empty. The blue 'register' button is at the bottom.

invalid password

A screenshot of a web browser displaying a registration form. The 'password' field contains 'password' and the 'confirm' field contains 'confirm'. Red error messages are visible below both fields, indicating they are invalid. The blue 'register' button is at the bottom.

password must match



Saved: 4/10/2026 6:57:43 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The register form uses HTML5 validation attributes (required, type="email", minlength, maxlength) so the browser blocks submission early and shows user-friendly messages. I also implemented sticky-form behavior for email and username by re-printing \$_POST values into the inputs, so if the server-side validation fails the user can correct the input without

≡ Task #2 (0.33 pts.) - JS Validation (validate() function)

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code related to the register page's `validate()` function
- Show examples of each validation message [username format, email format, password format, password doesn't match confirm](you may be able to capture this in one or few screenshots)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag

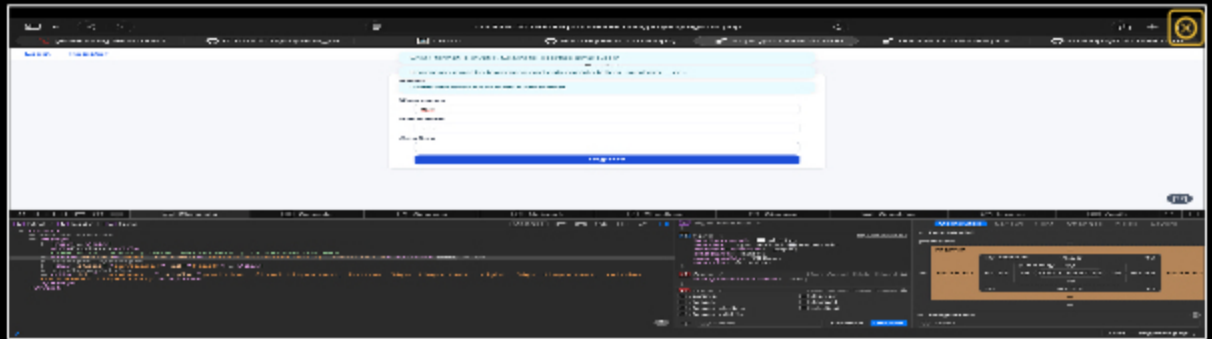
[illegible]

Code snippet

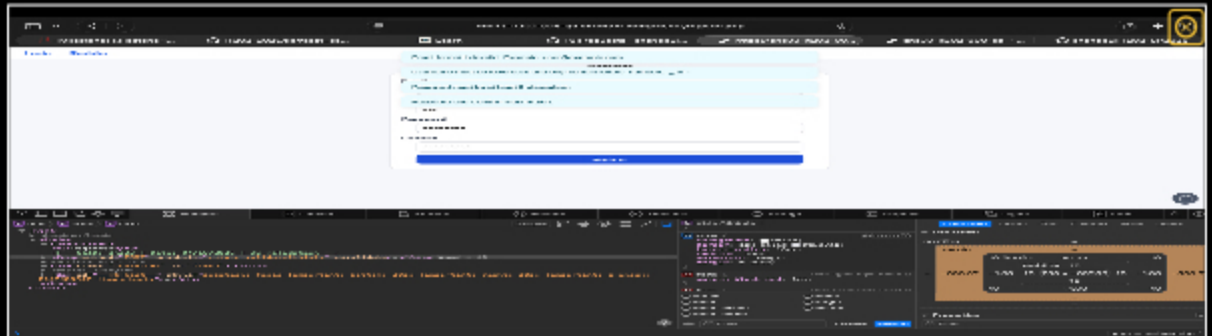
The screenshot shows a web browser window with a 'Your Email' form. The form has three input fields: 'Email', 'Password', and 'Confirm Password'. Below the fields is a blue 'Register' button. The browser's address bar shows the URL 'http://localhost:3000/register'. Below the browser window, a terminal window is open, displaying the output of a curl command. The output shows a successful response from the server, including a '200 OK' status and a 'Content-Type: application/json' header. The JSON body contains a 'message' field with the value 'User registered successfully' and a 'status' field with the value 'success'.

Invalid email

Invalid username



Invalid password



password must match

 Saved: 4/10/2026 6:56:24 PM

≡ Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

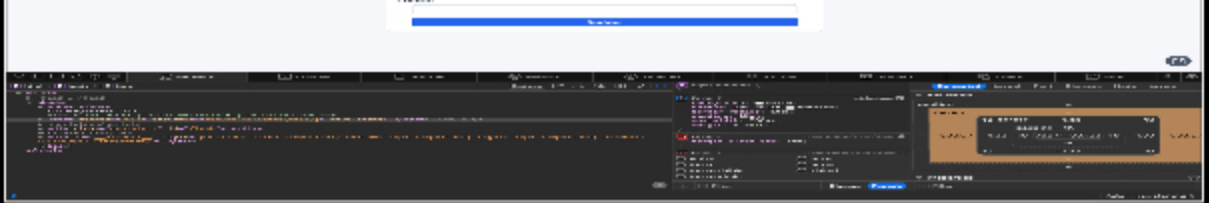
I added a validate(form) JavaScript function that runs on form submit. It checks the email format, enforces a username rule (lowercase letters/numbers plus _ or -), verifies the password is at least 8 characters, and confirms the password matches the confirm field. If any check fails, the function shows a clear message and returns false to prevent the form from submitting

 Saved: 4/10/2026 6:56:24 PM

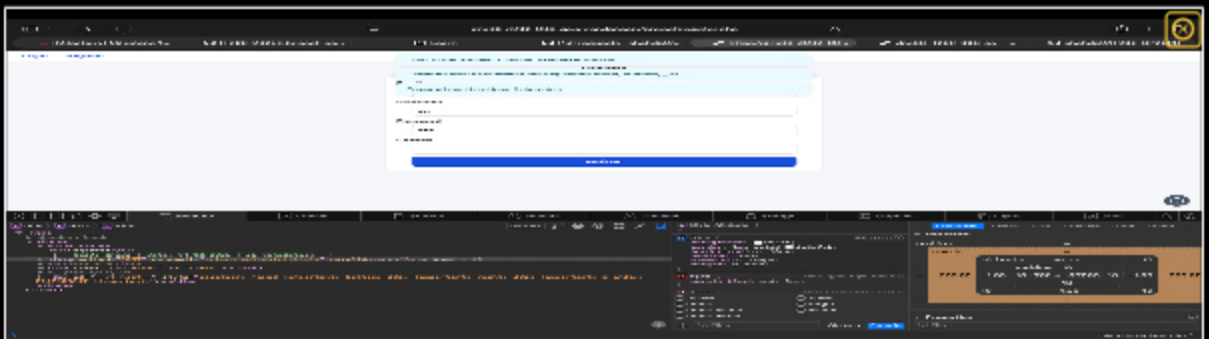
≡ Task #3 (0.33 pts.) - PHP Validation (steps before the DB call)

Progress: 100%

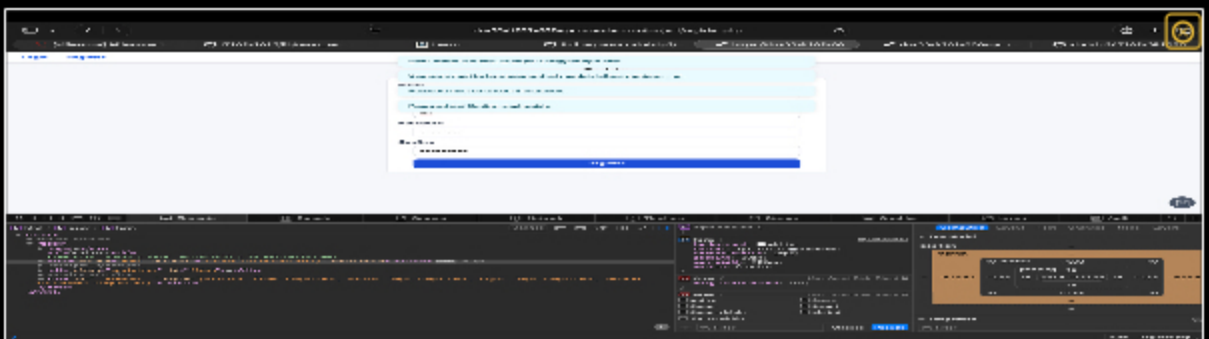
■ Part 1:



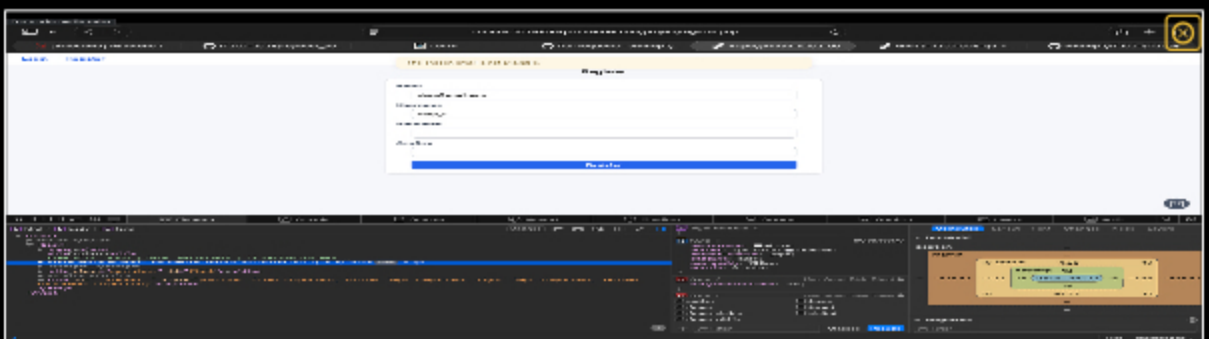
invalid username



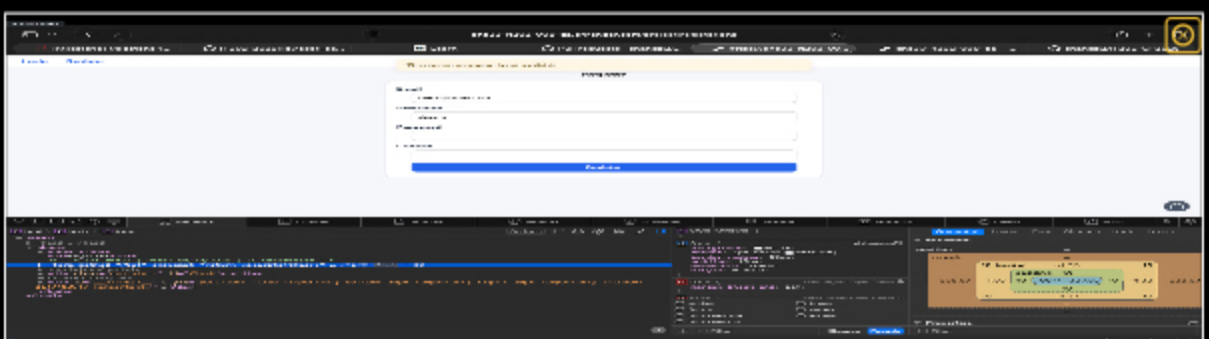
invalid password



password must match



email taken



username taken



Saved: 4/10/2026 7:16:25 PM

≡ Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The server re-validates all fields using helper functions (sanitize_email, is_valid_email, is_valid_username, etc.) before inserting into the database. If validation fails or the username/email is already taken, the server flashes an error message and does not write to the database.



Saved: 4/10/2026 7:16:25 PM

↔ Task #2 (0.67 pts.) - Related URLs

Progress: 100%

Details:

- Include the direct link to this file from the Milestone branch (should end in `.php` , zero credit if it does not)
- Include the render.com prod link to this file (Just grab the base prod url and manually enter the path to the file)
- Include the related pull request link for this feature

URL #1

https://github.com/shahdp2/IT202-SP2026/blob/Milestone1/public_html/project/register.php



URL

<https://github.com/shahdp2/>



URL #2

<https://dns33-it202-008-prod.onrender.com/project/register.php>



URL

<https://dns33-it202-008-prod.>



URL #3

<https://github.com/shahdp2/IT202-SP2026/pull/65>



URL

<https://github.com/shahdp2/>



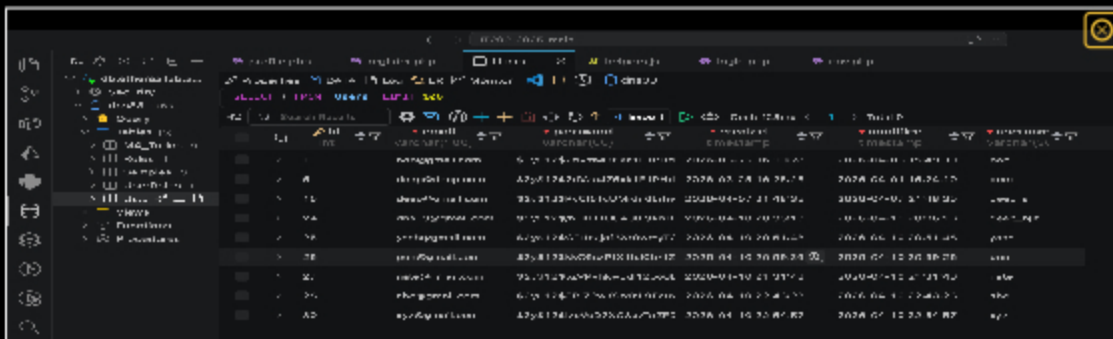
Saved: 4/10/2026 7:15:26 PM

📄 Task #3 (0.67 pts.) - Database - Users table

Progress: 100%

Details:

- Add a screenshot of the database view from vs code showing a valid registered user (preferably a recent one during evidence capturing)



id	username	email	password	created_at
1	admin	admin@localhost	\$2y\$12\$5c49130e12a2239d37963f40586e198b	2026-04-10 16:36:15
2	user1	user1@localhost	\$2y\$12\$5c49130e12a2239d37963f40586e198b	2026-04-10 16:36:15
3	user2	user2@localhost	\$2y\$12\$5c49130e12a2239d37963f40586e198b	2026-04-10 16:36:15
4	user3	user3@localhost	\$2y\$12\$5c49130e12a2239d37963f40586e198b	2026-04-10 16:36:15
5	user4	user4@localhost	\$2y\$12\$5c49130e12a2239d37963f40586e198b	2026-04-10 16:36:15
6	user5	user5@localhost	\$2y\$12\$5c49130e12a2239d37963f40586e198b	2026-04-10 16:36:15
7	user6	user6@localhost	\$2y\$12\$5c49130e12a2239d37963f40586e198b	2026-04-10 16:36:15
8	user7	user7@localhost	\$2y\$12\$5c49130e12a2239d37963f40586e198b	2026-04-10 16:36:15
9	user8	user8@localhost	\$2y\$12\$5c49130e12a2239d37963f40586e198b	2026-04-10 16:36:15
10	user9	user9@localhost	\$2y\$12\$5c49130e12a2239d37963f40586e198b	2026-04-10 16:36:15

image

 Saved: 4/10/2026 7:16:04 PM

Section #2: (2 pts.) Feature: User Will Be Able To Login To Their Account

Progress: 100%

≡ Task #1 (0.67 pts.) - Validation

Progress: 100%

Details:

- Show the relevant code snippets for each validation layer (html, js, php)
- Show the relevant demo of each validation layer (html, js, php)
- Briefly explain how the validation steps work at each layer

≡ Task #1 (0.33 pts.) - HTML Form/Validation

Progress: 100%

Details:

- System should allow the user to correct the error without wiping/clearing the form (for the PHP side this includes sticky-form logic to keep the username/email address entries)

Part 1:

Progress: 100%

Details:

- Show the code related to the login page's form (HTML validation)
- Show examples of each validation message (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)

```

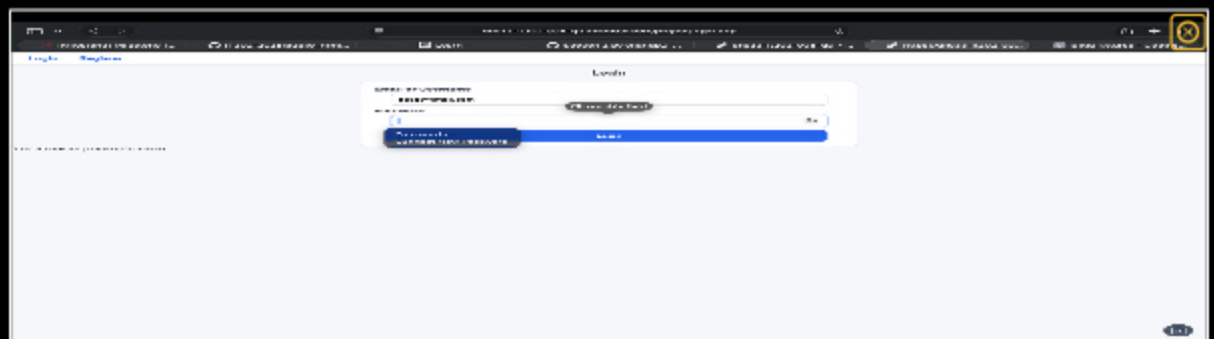
1 public_html > project > login.php > <!-- script
2 <script src="/assets/js/login.js"></script>
3 </head>
4 <body>
5 <div id="login">
6 <div id="login-form">
7 <div id="login-form">
8 <div id="login-form">
9 <div id="login-form">
10 <div id="login-form">
11 <div id="login-form">
12 <div id="login-form">
13 <div id="login-form">
14 <div id="login-form">
15 <div id="login-form">
16 <div id="login-form">
17 </div>

```

code snippet



Empty form

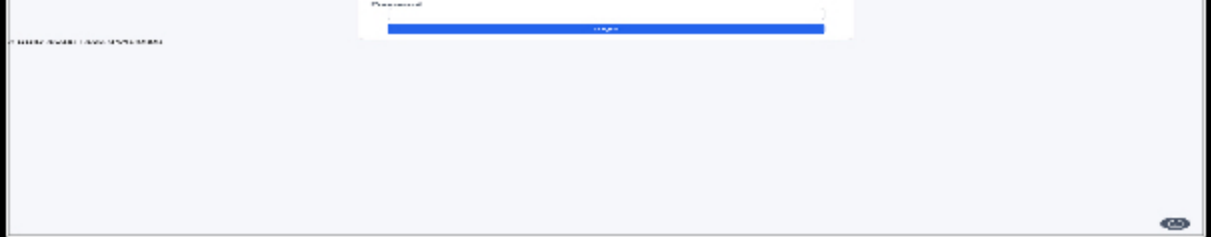


password empty

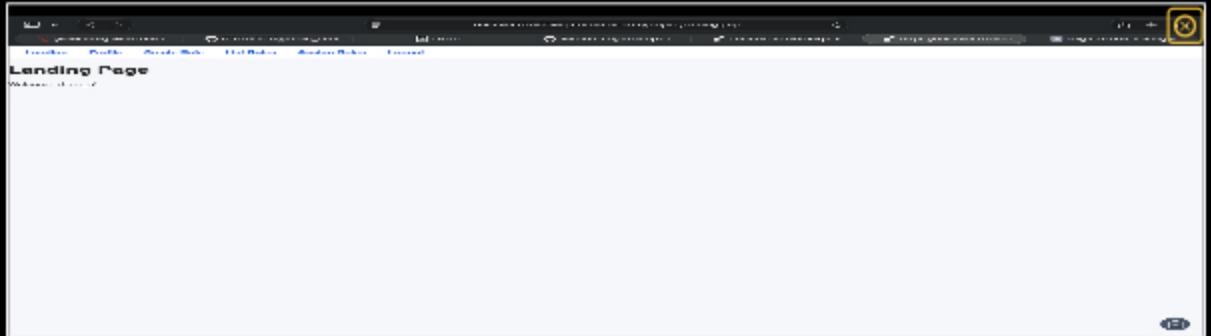


short password





wrong password



successfully logged into landing page



Saved: 4/10/2026 9:45:04 PM

≡, Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

I used built-in HTML form validation with required on both the email/username field and the password field, and minlength="8" on the password input. This prevents submitting the form when required fields are empty or the password is too short, so users get immediate browser feedback and can fix the input right away. I also used a sticky value for the email/username field (value="") so if the page reloads due to a server-side error, the user doesn't have to retype it.



Saved: 4/10/2026 9:45:04 PM

≡ Task #2 (0.33 pts.) - JS Validation (validate() function)

Progress: 100%

📄 Part 1:

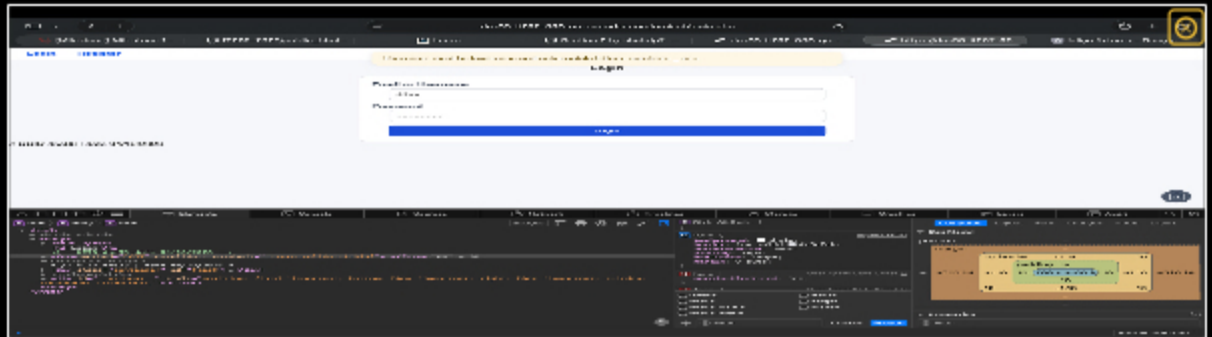
Progress: 100%

Details:

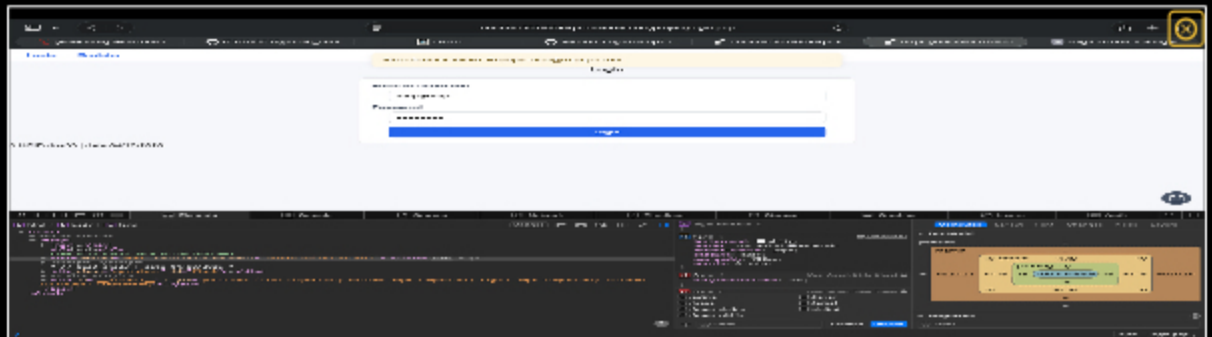
- Show the code related to the login page's validate() function

- Show examples of each validation message [username format, email format, password format] (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag

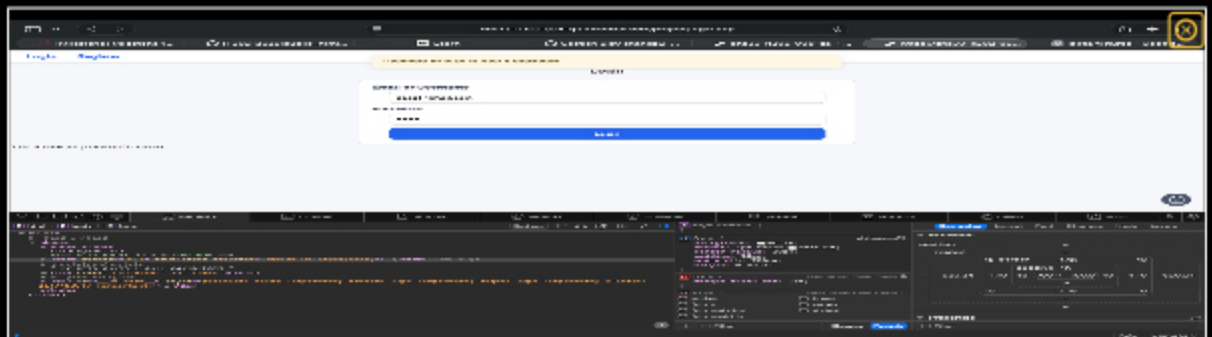
code snippet



invalid username



invalid email



short password

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

I added a `validate()` function that runs before the form submits. It checks that the password is at least 8 characters, then determines whether the login input looks like an email (contains `@`) or a username. If it's an email, it performs a basic email format check; if it's a username, it enforces the allowed pattern (lowercase letters, numbers, `_` and `-`). If any rule fails, it shows a flash message and returns false to stop submission so the user can correct the input immediately.

Task #3 (0.33 pts.) - PHP Validation (steps before the DB call)

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code related to the login page's PHP validation
- Show examples of each validation message [username format, email format, password format, invalid password] (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag and temporarily override the `validate()` function or use the provided helper script in the instructions
- Include the outputs related to user doesn't exist and php-side password mismatch (the one that checks against the DB hash)

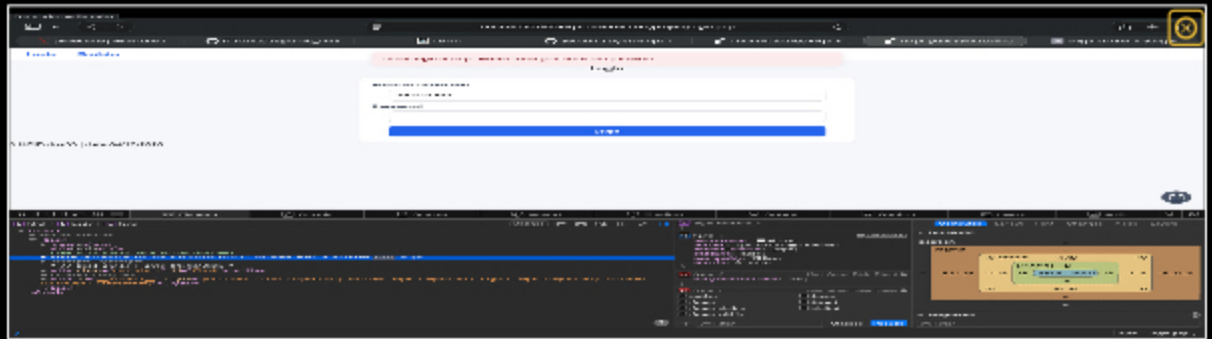
```

1  <?php
2  // ... (previous code) ...
3  // Validation functions
4  function validate($username, $password) {
5      // Password length check
6      if (strlen($password) < 8) {
7          return 'Password must be at least 8 characters long.';
8      }
9      // Username or email check
10     if (!preg_match('/^[a-zA-Z0-9_@-]+$/', $username)) {
11         return 'Username or email is invalid.';
12     }
13     // ... (other validation logic) ...
14 }
15
16 // ... (login logic) ...
17 if (!validate($username, $password)) {
18     // Show error message
19 }
20
21 // ... (password mismatch logic) ...
22 if ($password != password_hash($password, PASSWORD_DEFAULT)) {
23     // Show error message
24 }
25
26 // ... (user not found logic) ...
27 if (!User::exists($username)) {
28     // Show error message
29 }
30
31 // ... (end of script) ...

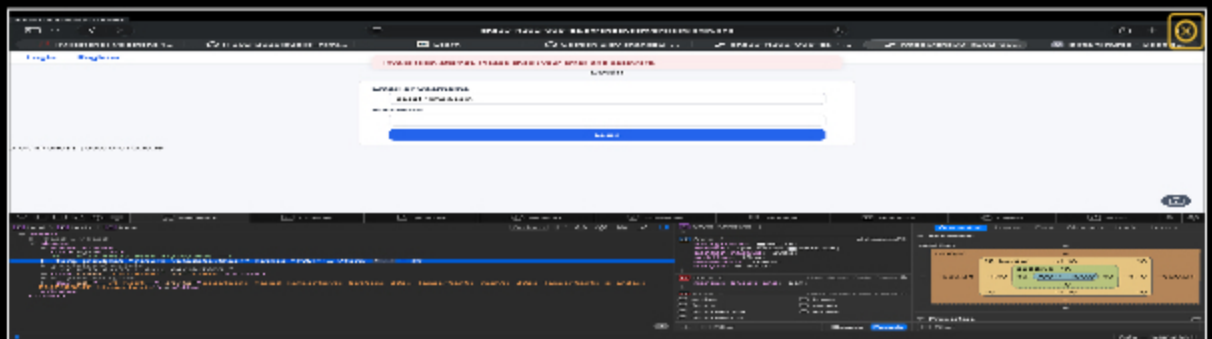
```




short password



username doesn't exist



wrong password

 Saved: 4/10/2026 10:18:00 PM

≡, Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

On submit, the server validates inputs before any database query. It checks that the email/username field is not empty, then decides whether the value is an email (contains @) or a username and validates the format accordingly. It also checks that the password is not empty and meets the minimum length rule. Only if all checks pass does it query the Users table (by email OR username). If a user record is found, it verifies the password using `password_verify()` against the stored hash. If the user doesn't exist or the password hash check fails, it shows a user-friendly "Invalid login attempt" message.

 Saved: 4/10/2026 10:18:00 PM

Task #2 (0.67 pts.) - Related URLs

Progress: 100%

Details:

- Include the direct link to this file from the Milestone branch (should end in `.php` , , zero credit if it does not)
- Include direct link to the file on Render.com Prod (Just grab the base prod url and manually enter the path to the file)
- Include the related pull request link for this feature

URL #1

https://github.com/shahdp2/IT202-SP2026/blob/Milestone1/public_html/project/login.php



URL

<https://github.com/shahdp2/>



URL #2

<https://dns33-it202-008-prod.onrender.com/project/login.php>



URL

<https://dns33-it202-008-prod.onrender.com/>



URL #3

<https://github.com/shahdp2/IT202-SP2026/pull/67>



URL

<https://github.com/shahdp2/>



Saved: 4/10/2026 10:23:07 PM

Task #3 (0.67 pts.) - User Session Evidence

Progress: 100%

Details:

- From the render.com logs (Logs tab), capture the session `err`
- It should show the id, username, email, and roles

```
10:25:32.70 [info] [Sat Apr 11 02:25:31.270530 2026] [php:error] [pid 17:pid 17] [client 114.4834] Session: array ( 'Flash' => 'array ( 'id' => 'array ( 'id' => 15, 'email' => 'deep@gmail.com', 'username' => 'deep_s', 'roles' => 'array ( 'name' => 'Admin', ' ' )', 'referer' => 'https://dns33-it202-008-prod.onrender.com/project/login.php'
```

session evidence



Saved: 4/10/2026 10:29:11 PM

Section #3: (1 pt.) Feature: User Will Be Able To Logout

Progress: 100%

Task #1 (1 pt.) - Evidence

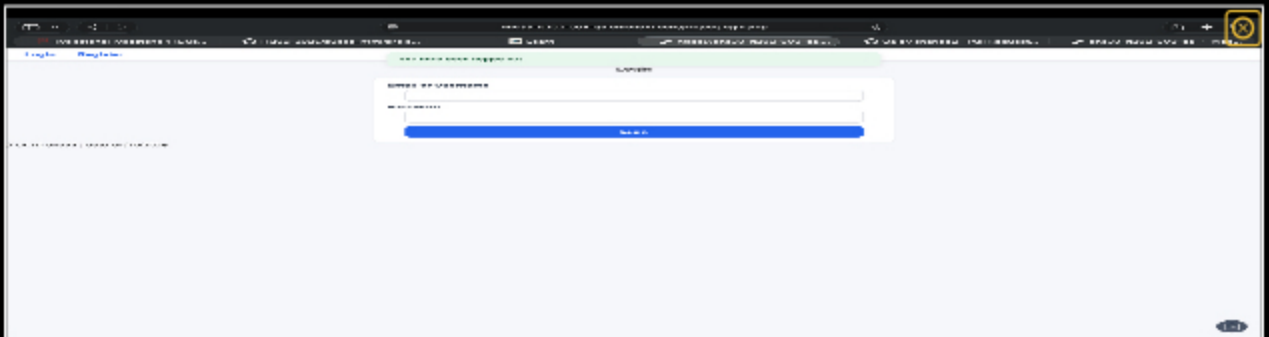
Progress: 100%

Part 1:

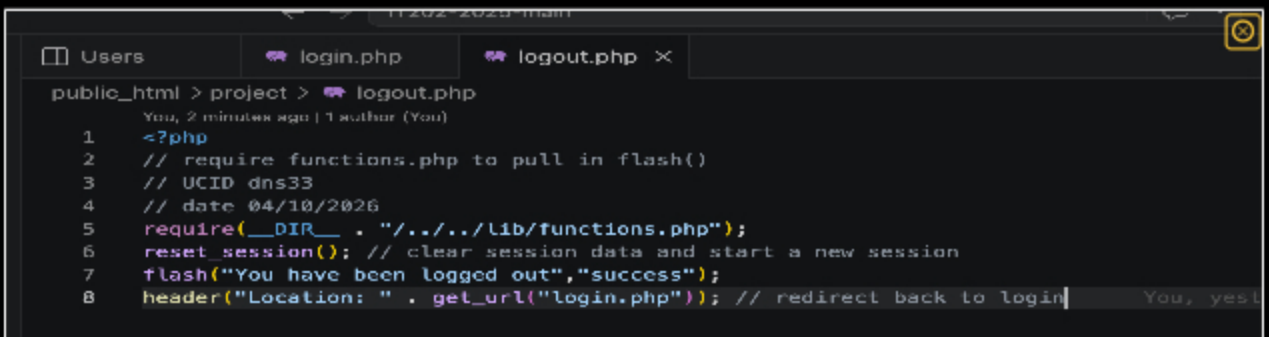
Progress: 100%

Details:

- Show the successful logout message (visit the proper file on Render.com qa after a manual deploy)
- Show the code snippet of the logout page



logout snippet



snippet

Saved: 4/10/2026 10:39:49 PM

Part 2:

Progress: 100%

Details:

- Include the pull request url for this feature

URL #1

<https://github.com/shahdp2/IT202-SP2026/pull/68>


URL

<https://github.com/shahdp2/IT202-SP2026/pull/68>


Saved: 4/10/2026 10:39:49 PM

Part 3:

Progress: 100%

Details:

- Briefly explain how the logout process works and how the user is officially logged off

Your Response:

When the user clicks Logout, the app runs `logout.php`. That file calls `reset_session()`, which clears the current session data (removes `$_SESSION` values like `$_SESSION["user"]`), destroys the old session, and starts a fresh empty session. After that, it sets a flash message like "You have been logged out" and redirects the user to `login.php`.

The user is officially logged off because their session no longer contains the logged-in user data (`$_SESSION["user"]` is gone). Since `is_logged_in()` checks whether `$_SESSION["user"]` exists, protected pages will now treat them as logged out and redirect them back to the login page.



Saved: 4/10/2026 10:39:49 PM

Section #4: (2 pts.) Feature: Security Rules

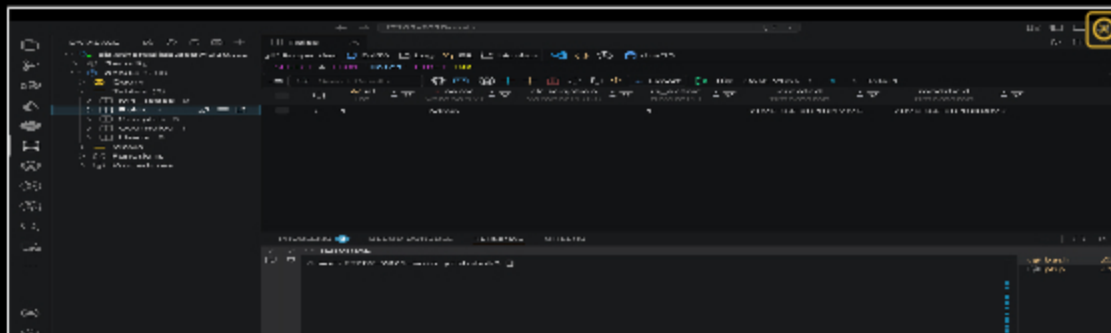
Progress: 100%

Task #1 (1 pt.) - Database Tables

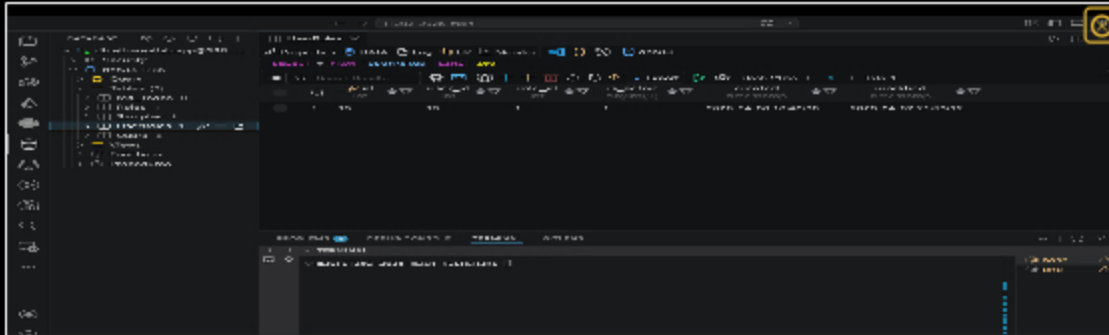
Progress: 100%

Details:

- From the vs code mysql tool, show both the ``Roles`` and ``User``



Roles



User Roles



Saved: 4/9/2026 6:52:21 PM

Task #2 (1 pt.) - Demo Security Rules

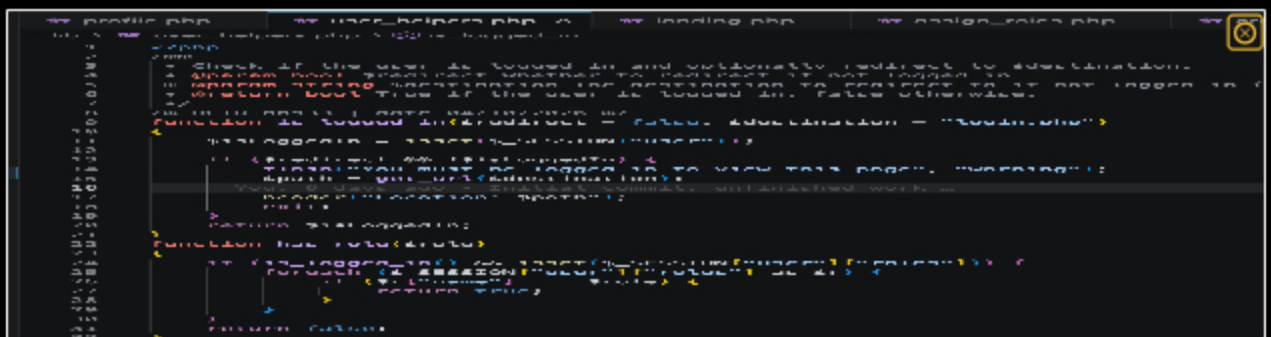
Progress: 100%

Part 1:

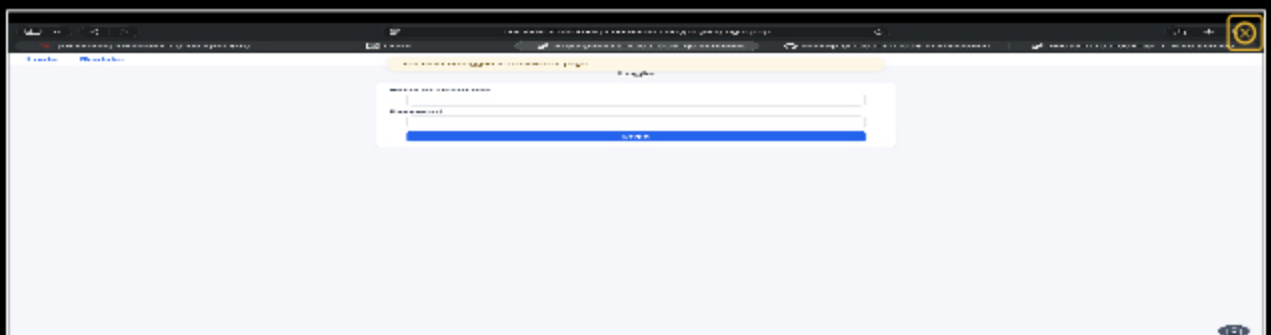
Progress: 100%

Details:

- Show the message related to needing to be logged in (i.e., try to
- Show the message related to not having the proper permission/role
- Include a snippet of the login check function
- Include a snippet of the role check function



code snippet



Logged-out user manually type /project/profile.php gets flash warning and is redirected to login



Non-admin user manually type /project/admin/list_roles.php gets permission warning and is redirected to landing

 Saved: 4/11/2026 12:29:12 PM

↪ Part 2:

Progress: 100%

Details:

- Include the pull request url for this feature

URL #1

<https://github.com/shahdp2/IT202-SP2026/pull/73>



URL

<https://github.com/shahdp2/IT202-SP2026/pull/73>

 Saved: 4/11/2026 12:29:12 PM

≡ Part 3:

Progress: 100%

Details:

- Briefly explain how the login check code works
- Briefly explain how the role check code works

Your Response:

Login check explanation `is_logged_in()` checks whether `$_SESSION["user"]` exists. If `$redirect` is true and the user is not logged in, it flashes a warning message and redirects to the login page using `get_url()`, then stops the script with `die()` so the protected page cannot load.

Role check explanation `has_role()` first ensures the user is logged in, then loops through `$_SESSION["user"]["roles"]` and returns true if any role name matches the required role (like "Admin"). If no match is found, it returns false so the calling page can block access and show a permission message.

 Saved: 4/11/2026 12:29:12 PM

Section #5: (2 pts.) Feature: User Profile

Progress: 100%

≡ Task #1 (1 pt.) - Validation

Progress: 100%

Details:

- Show the relevant code snippets for each validation layer (html, js, php)
- Show the relevant demo of each validation layer (html, js, php)
- Briefly explain how the validation steps work at each layer

≡ Task #1 (0.33 pts.) - HTML Form/Validation

Progress: 100%

Details:

- System should allow the user to correct the error without wiping/clearing the form (for the PHP side this includes sticky-form logic to keep the username/email address entries)

Part 1:

Progress: 100%

Details:

- Show the code related to the profile page's form (HTML validation)
- Show examples of each validation message (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)

snippet



empty email

The screenshot shows a web application interface with a navigation bar at the top containing links: Home, Profile, Account, and Logout. Below the navigation bar, there is a form titled "Create New Account". The form contains several input fields: "Email", "First Name", "Last Name", "Phone Number", "New Password", and "Confirm Password". The "Email" field is currently empty. At the bottom of the form, there is a blue button labeled "Create Account".

invalid email

The screenshot shows the same "Create New Account" form as before. In this instance, the "Email" field contains the text "invalid_email". The "New Password" and "Confirm Password" fields are also filled with "password123". The blue "Create Account" button is visible at the bottom of the form.

invalid username

The screenshot shows the "Create New Account" form. The "Email" field is filled with "example@example.com". The "First Name" field contains "John", and the "Last Name" field contains "Doe". The "Phone Number" field is filled with "1234567890". The "New Password" and "Confirm Password" fields both contain "password123". The blue "Create Account" button is at the bottom.

New and Confirm Password must match.

 Saved: 4/11/2026 1:17:56 PM

≡, Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

I used built-in HTML validation so the browser blocks bad input before the form submits. The Email field uses type="email" and required, so it can't be empty and must look like a real email format. The Username field is required with maxlength="30", so it can't be blank and it prevents overly long usernames. Since the inputs stay on the page, the user can fix the highlighted field and resubmit without losing what they already typed.

 Saved: 4/11/2026 1:17:56 PM

Task #2 (0.33 pts.) - JS Validation (validate() function)

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code related to the profile page's validate() function
- Show examples of each validation message [username format, email format, password format, password doesn't match confirm] (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag

```
187 // validate page JS
188 // after form validation JS, profile page JS, and login JS validation
189 // script
190 // 1. Form JS
191 // 2. Form JS
192 // 3. Form JS
193 // 4. Form JS
194 // 5. Form JS
195 // 6. Form JS
196 // 7. Form JS
197 // 8. Form JS
198 // 9. Form JS
199 // 10. Form JS
200 // 11. Form JS
201 // 12. Form JS
202 // 13. Form JS
203 // 14. Form JS
204 // 15. Form JS
205 // 16. Form JS
206 // 17. Form JS
207 // 18. Form JS
208 // 19. Form JS
209 // 20. Form JS
210 // 21. Form JS
211 // 22. Form JS
212 // 23. Form JS
213 // 24. Form JS
214 // 25. Form JS
215 // 26. Form JS
216 // 27. Form JS
217 // 28. Form JS
218 // 29. Form JS
219 // 30. Form JS
220 // 31. Form JS
221 // 32. Form JS
222 // 33. Form JS
223 // 34. Form JS
224 // 35. Form JS
225 // 36. Form JS
226 // 37. Form JS
227 // 38. Form JS
228 // 39. Form JS
229 // 40. Form JS
230 // 41. Form JS
231 // 42. Form JS
232 // 43. Form JS
233 // 44. Form JS
234 // 45. Form JS
235 // 46. Form JS
236 // 47. Form JS
237 // 48. Form JS
238 // 49. Form JS
239 // 50. Form JS
240 // 51. Form JS
241 // 52. Form JS
242 // 53. Form JS
243 // 54. Form JS
244 // 55. Form JS
245 // 56. Form JS
246 // 57. Form JS
247 // 58. Form JS
248 // 59. Form JS
249 // 60. Form JS
250 // 61. Form JS
251 // 62. Form JS
252 // 63. Form JS
253 // 64. Form JS
254 // 65. Form JS
255 // 66. Form JS
256 // 67. Form JS
257 // 68. Form JS
258 // 69. Form JS
259 // 70. Form JS
260 // 71. Form JS
261 // 72. Form JS
262 // 73. Form JS
263 // 74. Form JS
264 // 75. Form JS
265 // 76. Form JS
266 // 77. Form JS
267 // 78. Form JS
268 // 79. Form JS
269 // 80. Form JS
270 // 81. Form JS
271 // 82. Form JS
272 // 83. Form JS
273 // 84. Form JS
274 // 85. Form JS
275 // 86. Form JS
276 // 87. Form JS
277 // 88. Form JS
278 // 89. Form JS
279 // 90. Form JS
280 // 91. Form JS
281 // 92. Form JS
282 // 93. Form JS
283 // 94. Form JS
284 // 95. Form JS
285 // 96. Form JS
286 // 97. Form JS
287 // 98. Form JS
288 // 99. Form JS
289 // 100. Form JS
```

snippet - 1

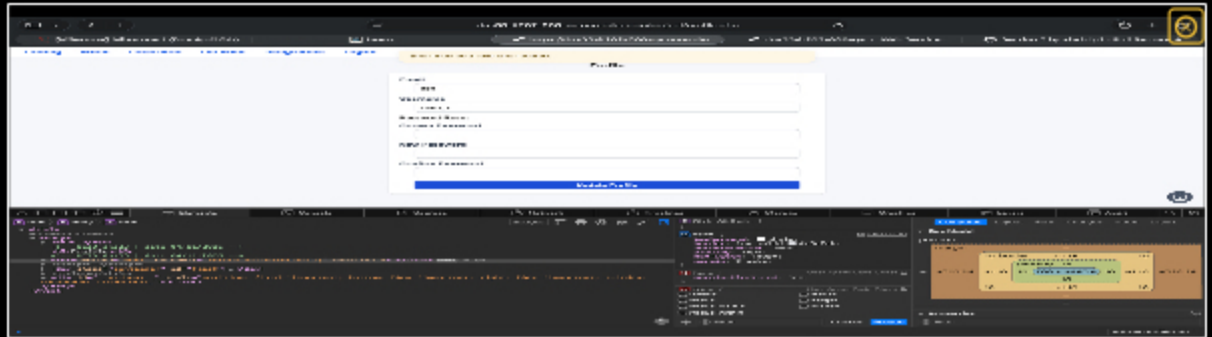
```
187 // validate page JS
188 // after form validation JS, profile page JS, and login JS validation
189 // script
190 // 1. Form JS
191 // 2. Form JS
192 // 3. Form JS
193 // 4. Form JS
194 // 5. Form JS
195 // 6. Form JS
196 // 7. Form JS
197 // 8. Form JS
198 // 9. Form JS
199 // 10. Form JS
200 // 11. Form JS
201 // 12. Form JS
202 // 13. Form JS
203 // 14. Form JS
204 // 15. Form JS
205 // 16. Form JS
206 // 17. Form JS
207 // 18. Form JS
208 // 19. Form JS
209 // 20. Form JS
210 // 21. Form JS
211 // 22. Form JS
212 // 23. Form JS
213 // 24. Form JS
214 // 25. Form JS
215 // 26. Form JS
216 // 27. Form JS
217 // 28. Form JS
218 // 29. Form JS
219 // 30. Form JS
220 // 31. Form JS
221 // 32. Form JS
222 // 33. Form JS
223 // 34. Form JS
224 // 35. Form JS
225 // 36. Form JS
226 // 37. Form JS
227 // 38. Form JS
228 // 39. Form JS
229 // 40. Form JS
230 // 41. Form JS
231 // 42. Form JS
232 // 43. Form JS
233 // 44. Form JS
234 // 45. Form JS
235 // 46. Form JS
236 // 47. Form JS
237 // 48. Form JS
238 // 49. Form JS
239 // 50. Form JS
240 // 51. Form JS
241 // 52. Form JS
242 // 53. Form JS
243 // 54. Form JS
244 // 55. Form JS
245 // 56. Form JS
246 // 57. Form JS
247 // 58. Form JS
248 // 59. Form JS
249 // 60. Form JS
250 // 61. Form JS
251 // 62. Form JS
252 // 63. Form JS
253 // 64. Form JS
254 // 65. Form JS
255 // 66. Form JS
256 // 67. Form JS
257 // 68. Form JS
258 // 69. Form JS
259 // 70. Form JS
260 // 71. Form JS
261 // 72. Form JS
262 // 73. Form JS
263 // 74. Form JS
264 // 75. Form JS
265 // 76. Form JS
266 // 77. Form JS
267 // 78. Form JS
268 // 79. Form JS
269 // 80. Form JS
270 // 81. Form JS
271 // 82. Form JS
272 // 83. Form JS
273 // 84. Form JS
274 // 85. Form JS
275 // 86. Form JS
276 // 87. Form JS
277 // 88. Form JS
278 // 89. Form JS
279 // 90. Form JS
280 // 91. Form JS
281 // 92. Form JS
282 // 93. Form JS
283 // 94. Form JS
284 // 95. Form JS
285 // 96. Form JS
286 // 97. Form JS
287 // 98. Form JS
288 // 99. Form JS
289 // 100. Form JS
```

snippet - 2

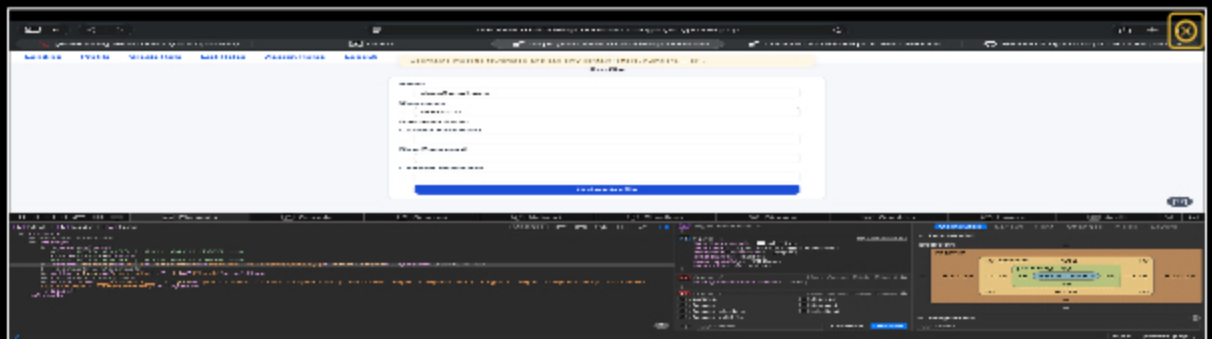
```
187 // validate page JS
188 // after form validation JS, profile page JS, and login JS validation
189 // script
190 // 1. Form JS
191 // 2. Form JS
192 // 3. Form JS
193 // 4. Form JS
194 // 5. Form JS
195 // 6. Form JS
196 // 7. Form JS
197 // 8. Form JS
198 // 9. Form JS
199 // 10. Form JS
200 // 11. Form JS
201 // 12. Form JS
202 // 13. Form JS
203 // 14. Form JS
204 // 15. Form JS
205 // 16. Form JS
206 // 17. Form JS
207 // 18. Form JS
208 // 19. Form JS
209 // 20. Form JS
210 // 21. Form JS
211 // 22. Form JS
212 // 23. Form JS
213 // 24. Form JS
214 // 25. Form JS
215 // 26. Form JS
216 // 27. Form JS
217 // 28. Form JS
218 // 29. Form JS
219 // 30. Form JS
220 // 31. Form JS
221 // 32. Form JS
222 // 33. Form JS
223 // 34. Form JS
224 // 35. Form JS
225 // 36. Form JS
226 // 37. Form JS
227 // 38. Form JS
228 // 39. Form JS
229 // 40. Form JS
230 // 41. Form JS
231 // 42. Form JS
232 // 43. Form JS
233 // 44. Form JS
234 // 45. Form JS
235 // 46. Form JS
236 // 47. Form JS
237 // 48. Form JS
238 // 49. Form JS
239 // 50. Form JS
240 // 51. Form JS
241 // 52. Form JS
242 // 53. Form JS
243 // 54. Form JS
244 // 55. Form JS
245 // 56. Form JS
246 // 57. Form JS
247 // 58. Form JS
248 // 59. Form JS
249 // 60. Form JS
250 // 61. Form JS
251 // 62. Form JS
252 // 63. Form JS
253 // 64. Form JS
254 // 65. Form JS
255 // 66. Form JS
256 // 67. Form JS
257 // 68. Form JS
258 // 69. Form JS
259 // 70. Form JS
260 // 71. Form JS
261 // 72. Form JS
262 // 73. Form JS
263 // 74. Form JS
264 // 75. Form JS
265 // 76. Form JS
266 // 77. Form JS
267 // 78. Form JS
268 // 79. Form JS
269 // 80. Form JS
270 // 81. Form JS
271 // 82. Form JS
272 // 83. Form JS
273 // 84. Form JS
274 // 85. Form JS
275 // 86. Form JS
276 // 87. Form JS
277 // 88. Form JS
278 // 89. Form JS
279 // 90. Form JS
280 // 91. Form JS
281 // 92. Form JS
282 // 93. Form JS
283 // 94. Form JS
284 // 95. Form JS
285 // 96. Form JS
286 // 97. Form JS
287 // 98. Form JS
288 // 99. Form JS
289 // 100. Form JS
```



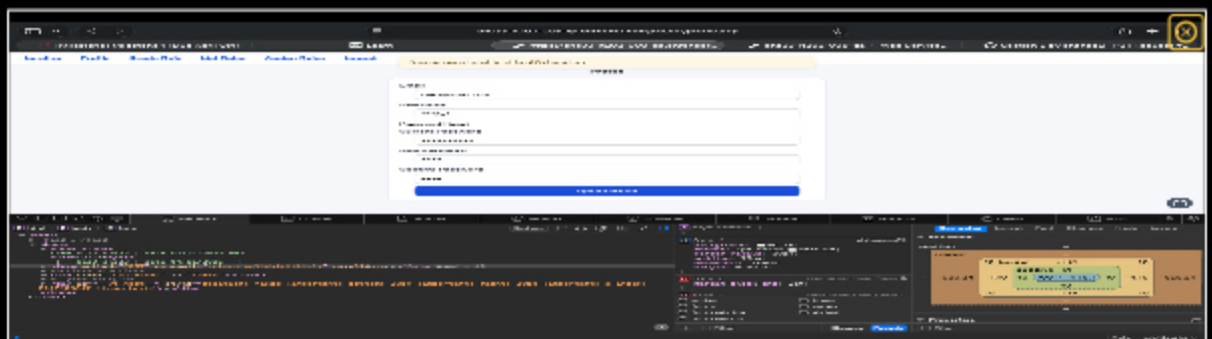
snippet - 3



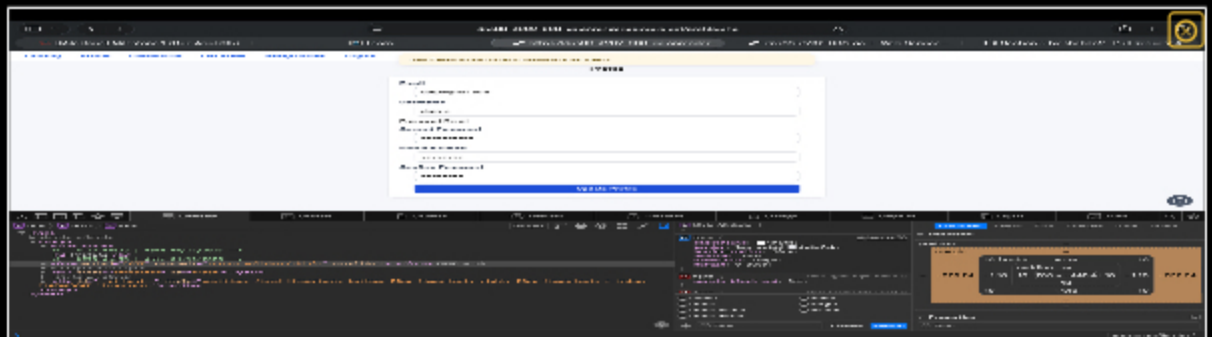
invalid email



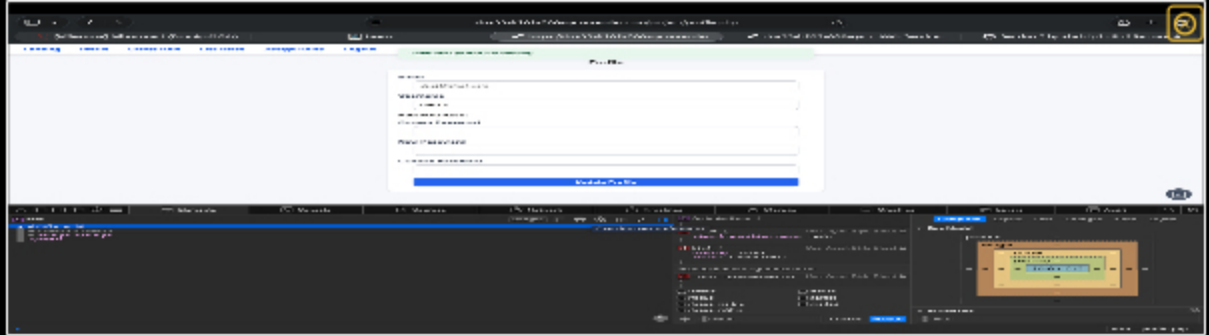
invalid username



short password



New and Confirm Password must match



successfully changed password/ user profile



Saved: 4/11/2026 1:28:17 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

My `validate()` function checks the form before submitting. It first validates the email and username format and shows a flash message if either is invalid. For password changes, it only runs password checks if the user filled any password fields; then it requires all three password inputs, checks the new password length, and confirms the new password matches the confirm field. If any check fails, it shows a flash warning and returns false to block the form submission.



Saved: 4/11/2026 1:28:17 PM

Task #3 (0.33 pts.) - PHP Validation (steps before the DB call)

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code related to the profile page's PHP validation
- Show examples of each validation message [username format, email format, password format, invalid password, password doesn't match confirm, username taken, email taken] (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag and temporarily override the `validate()` function or use the provided helper script in the instructions

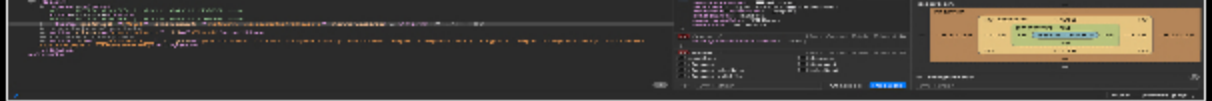
snip pet - 1



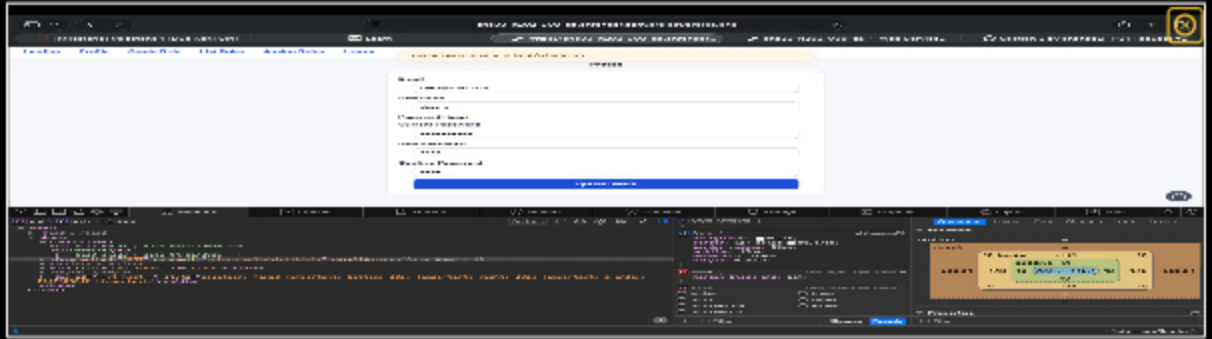
snip pet - 2

snippet - 3

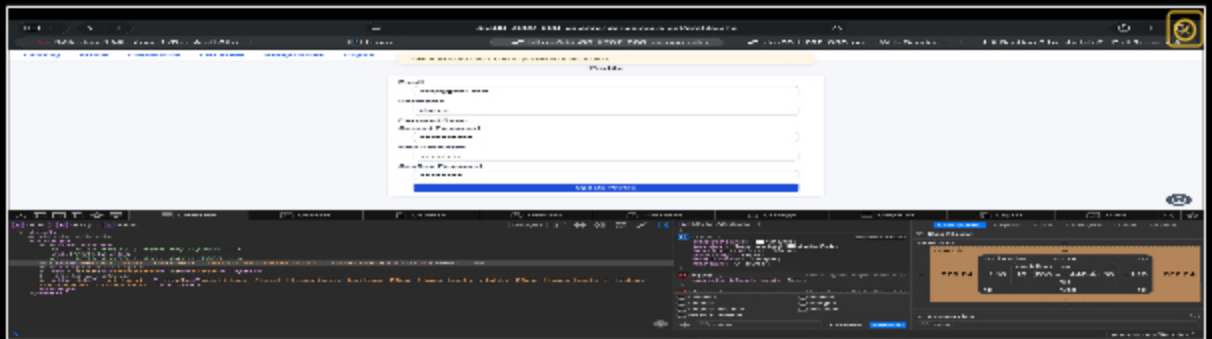
invalid email



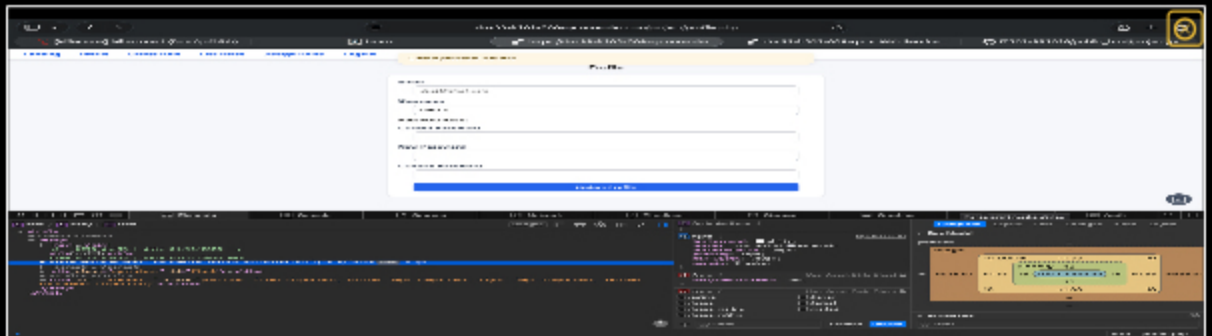
invalid username



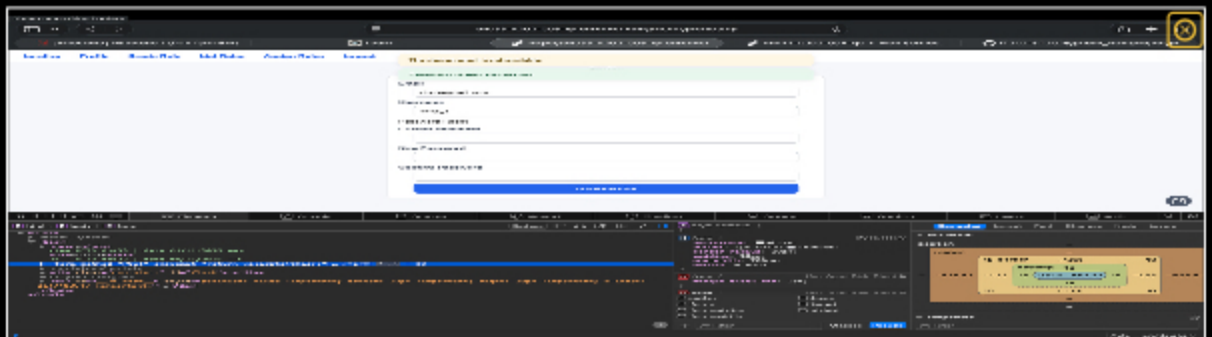
short password



new password and confirm password must match

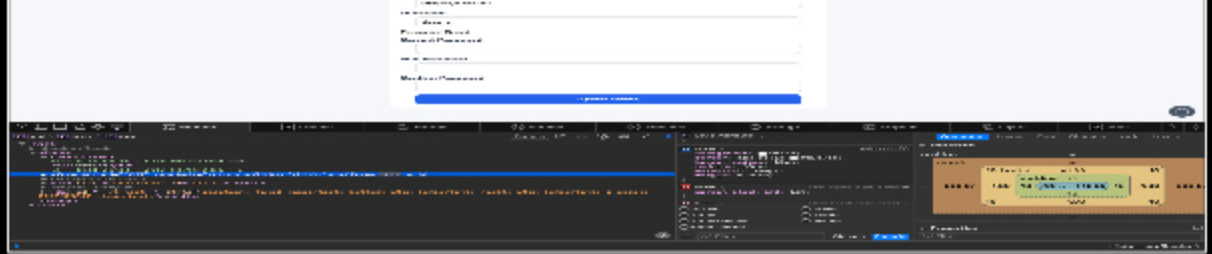


current password wrong



email already used by another user





username already used by another user



All users in DB so far

 Saved: 4/11/2026 1:49:05 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

I validate profile updates in two steps: first I sanitize/validate the new email and enforce the username format using helper functions. If either fails, I flash a specific message and skip the DB update. For password changes, I only proceed when all three fields (current/new/confirm) are provided; then I check new vs confirm, enforce the password rule (min length), and finally verify the current password against the stored hash before updating. Duplicate email/username updates are caught from the database via PDOException and converted into a user-friendly flash message.

 Saved: 4/11/2026 1:49:05 PM

Task #2 (1 pt.) - Related URLs

Progress: 100%

Details:

- Include the direct link to this file from the Milestone branch (should end in

.php , , zero credit if it does not)

- Include direct link to the file on Render.com Prod (Just grab the base prod url and manually enter the path to the file)
- Include the related pull request link for this feature

URL #1

https://github.com/shahdp2/IT202-SP2026/blob/Milestone1/public_html/project/profile.php



URL

<https://github.com/shahdp2/>



URL #2

<https://dns33-it202-008-prod.onrender.com/project/profile.php>



URL

<https://dns33-it202-008-prod.onrender.com/project/profile.php>



URL #3

<https://github.com/shahdp2/IT202-SP2026/pull/74>



URL

<https://github.com/shahdp2/>



Saved: 4/11/2026 1:36:21 PM

Section #6: (1 pt.) Misc

Progress: 100%

Task #1 (0.33 pts.) - Github Details

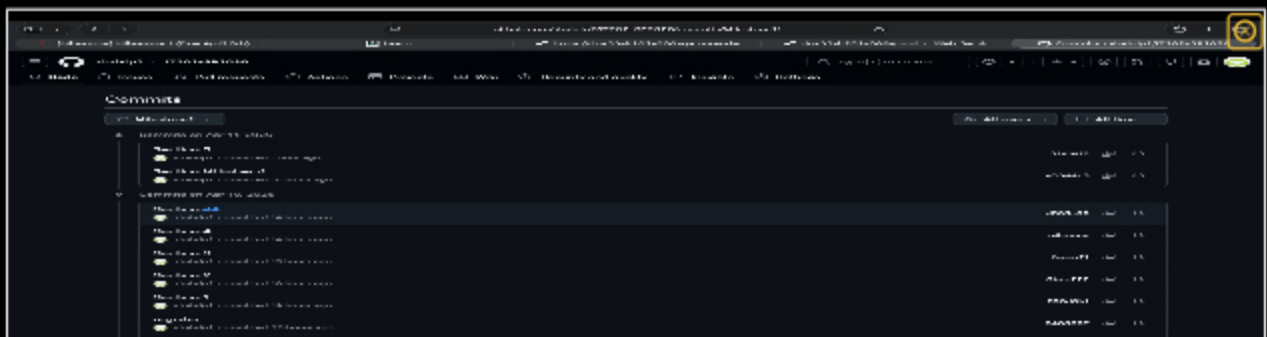
Progress: 100%

Part 1:

Progress: 100%

Details:

From the Commits tab of the Pull Request screenshot the commit history (it may not be possible to capture all, just grab a reasonable chunk)



commit history



Saved: 4/11/2026 1:58:50 PM

Part 2:

Progress: 100%

Details:

I learned how to build a complete authentication flow in PHP with proper

validation at multiple layers (HTML, JavaScript, and PHP). I also learned why server-side validation is required even if HTML/JS validation exists, because users can bypass the browser checks. Another big takeaway was understanding session management: storing user data in `$_SESSION`, loading roles, and using helper functions like `is_logged_in()` and `has_role()` to protect pages. Finally, I learned how to deploy to Render and capture evidence from logs and screenshots to prove features work in the hosted QA environment.



Saved: 4/11/2026 2:04:50 PM

⇒ Task #2 (0.33 pts.) - What was the easiest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

The easiest part was building the HTML forms and adding basic HTML validation like `required`, `type="email"`, and `minlength`. It was also straightforward to style the pages consistently using the shared CSS and the navigation bar. Capturing screenshots for the HTML validation messages was simple because the browser automatically shows messages when required fields are missing or formatted incorrectly.



Saved: 4/11/2026 2:05:00 PM

⇒ Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

The hardest part was getting all validation layers to work correctly without conflicts—especially testing PHP validation when HTML/JS would block the form submission first. It also took time to understand session and redirect behavior, since redirects must happen before output is sent, and small mistakes can cause header warnings. Debugging role-based access was another challenge because it depends on both database data (Roles/UserRoles) and session state. Overall, coordinating sessions, redirects, and validation across Register/Login/Profile pages was the most complex part.

pages was the most complex part.



Saved: 4/11/2026 2:05:15 PM